Khulna University of Engineering & Technology (KUET)
Department of Computer Science and Engineering

CSE 4102 · Computer Graphics and Image Processing Laboratory
4th Year, 1st Term

The Ramen Shop

Project Report: A Japanese Ramen Village in OpenGL



The shop at night, rendered in real time by the project.

Submitted by
Rajorshi Das
Roll: 2107060
Department of Computer Science and Engineering, KUET

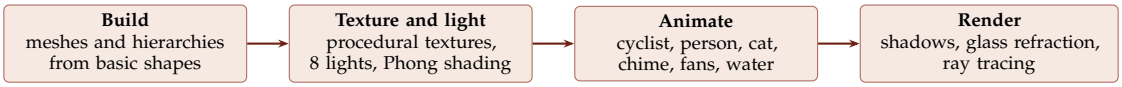
Submitted to
Md Tajmilur Rahman
Lecturer, Dept. of CSE, KUET

Md. Mubtashim Abrar Nihal
Lecturer, Dept. of CSE, KUET

Built with C++17 · OpenGL (fixed function + GLSL 1.20) · FreeGLUT · GLEW
October 2026

Abstract

The Ramen Shop is an interactive 3D Japanese village written from scratch in C++ and OpenGL. A two-storey ramen shop with a dining counter, an open kitchen and an upstairs tatami room stands on a cobblestone street with ten neighbouring houses, street lamps, trees, flower gardens, a lawn and a lake with ducks. A sky with a sun, a moon, stars and clouds changes between day and night; a cyclist, a walking person and a cat move through the street; and a ramen-cooking game can be played in the shop. The report states which features were proposed and which were implemented, and for every feature it explains *how it was implemented and which theory, formula or equation it uses*, with the code structure and a screenshot of the result. Every object and every texture is generated by the program; no model or image file is loaded.



Contents

1 Introduction **3**
1.1 Objectives . . . 3
1.2 How to read this report . . . 3
2 Proposed Features and What Was Implemented **3**
3 Tools, Libraries and Sources Followed **3**
4 Scene Layout and Program Structure **3**
4.1 Modules and the frame loop . . . 3
5 Implementation of Each Feature **3**
5.1 3D modelling from basic shapes . . . 6
5.2 Transformations and hierarchical modelling . . . 6
5.3 Camera, projection and mouse picking . . . 6
5.4 Illumination: ambient, diffuse and specular light . . . 7
5.5 Shading: Gouraud and Phong . . . 9
5.6 Textures and texture mapping . . . 10
5.7 Skybox, sun, moon, stars, clouds and day / night . . . 11
5.8 Blending, transparency, emission and fog . . . 12
5.9 Shadows . . . 13
5.10 Glass refraction (screen space) . . . 14
5.11 Ray tracing . . . 14
5.12 Animation and the characters . . . 16
5.13 Environment: trees, village, lake and the shop . . . 17
5.14 Interaction and the ramen cooking game . . . 17
6 Results **19**
7 Problems, Limitations and Future Work **21**
8 Conclusion **21**

1 Introduction

A small ramen-ya sits on a quiet Japanese street. Its two-storey wooden shop has a dining counter with an open kitchen, two small tables, a cosy tatami room upstairs, a street with sidewalks, cherry and maple trees, and a lake with ducks behind it. This project builds that whole place as an interactive, real-time **OpenGL** program. The viewer can fly around it with seven cameras, switch every lamp on and off, change between day and night, click doors open and play a small ramen-cooking game.

Everything is made in code. Every object is built from a few basic shapes (cube, cylinder / cone, sphere, torus, lathed and grid surfaces) and every texture is painted by the program itself at start-up. Lighting uses the ambient, diffuse and specular terms of the Phong model with per-vertex (Gouraud) and per-pixel (Phong) shading. On top of that come shadows, a refractive glass, a small CPU ray tracer, and many moving things.

1.1 Objectives

1. Model a believable multi-part scene (shop, upstairs room, street, nature) from basic shapes and 3D transformations.
2. Light it with the ambient, diffuse and specular terms, with several point, spot and directional lights that can each be switched on and off.
3. Apply procedurally generated textures with correct mapping, and shade with both Gouraud and Phong shading.
4. Add advanced effects: shadows (projected and shadow-mapped), screen-space refraction, and real ray tracing.
5. Make the scene alive and explorable: animation, several cameras, mouse and keyboard interaction, and a small game.

1.2 How to read this report

Section 2 lists what was proposed and what was implemented. Section 3 lists the tools and the sources followed. Section 4 describes the layout and the structure of the program. Section 5 is the main part: each feature has a *Theory* part (formulas and equations), an *Implementation* part (what the code does, with function names and numbers) and a *Result* part (screenshots from the running program). Section 6 summarises the results and Section 7 the problems and limitations.

2 Proposed Features and What Was Implemented

Table 1 compares the features of the proposal with the final project and gives the section that explains each one.

3 Tools, Libraries and Sources Followed

Table 3 lists the source or reference followed for each technique. Library functions are used as documented; the scene, the shaders, the textures, the ray tracer and the animation code are implemented in the project itself. The report layout of the Beach Day project report was used only as a guide to the structure of this report.

4 Scene Layout and Program Structure

One world coordinate system is used: x to the right, y up and z toward the street (the front of the shop); one unit is about one metre. The ground floor is 9.8×7.8 units and 3.3 high; the upper floor adds 2.0 and the gable roof rises another 2.2. The ground plane is a lawn 240 units wide with a lake at $(0, -24)$; the road and two sidewalks run along x at $z \approx 7$ to 17.

4.1 Modules and the frame loop

The program is split into small modules (Table 5). At start-up it opens the window (double buffer, depth, stencil), paints 13 textures and builds mipmaps, sets up 8 lights and fog, compiles the Phong shader and creates the shadow-map and glass framebuffers. Every frame it follows the pipeline of Figure 2.

5 Implementation of Each Feature

Proposed feature	Status	How it appears in the final project	Sec.
Translation	Implemented	cyclist, person, cat, ducks, sliding doors, steam, falling petals	5.2
Rotation	Implemented	wheels, cranks, fan blades, hinged door, chime pendulums, pivot sway of trees	5.2
Scaling	Implemented	planks from cubes, ellipsoids, cat $\times 1.8$, bicycle $\times 1.4$, user scaling of objects	5.2
Perspective camera	Implemented	45° (62° in game); seven cameras	5.3
Orthographic view	Adapted	used for the game HUD, order board and title screen, not for inspecting the scene	5.3
Hierarchical modelling	Implemented	cyclist, person, cat, wind chime, trees, lanterns	5.2
Lighting: ambient, diffuse, specular	Implemented	8 lights (directional, point, spot); each term can be switched off	5.4
Flat shading	Not implemented	only smooth shading is used (Gouraud or Phong)	5.5
Gouraud shading	Implemented	fixed-function path, key `G`	5.5
Phong shading	Implemented	own GLSL fragment shader, per pixel, 8 lights	5.5
Skybox	Implemented	gradient dome, sun, moon, stars, clouds, day / night	5.7
Textures	Implemented	13 procedural textures (grass, wood, tile, tatami, ...), mipmaps	5.6
Features added beyond the proposal			
Shadows	Implemented	stencil projected shadows, soft shadow map with PCF	5.9
Glass refraction	Implemented	screen-space refraction of the tumbler	5.10
Ray tracing	Implemented	CPU ray tracer with BVH: reflection and refraction (pots, bowls, ball, mirror)	5.11
Animated characters	Implemented	cyclist with inverse kinematics, walking person, cat state machine	5.12
Environment	Implemented	trees by recursion, flowers, lake, ducks, 10 houses, street, fireflies	5.13
Cooking game	Implemented	state machine, scoring, HUD	5.14

Table 1: Proposed features and their final status.

Language, build	C++17, Visual Studio (MSBuild), x64 Release / Debug; about 15 000 lines in 18 source files and 17 headers
Graphics API	OpenGL fixed-function pipeline plus GLSL 1.20 shaders (Phong lighting, glass refraction, ray-trace compositing); framebuffer objects and float textures (OpenGL 3.0)
Libraries	FreeGLUT (window, input), GLEW (extensions), GLU (quadrics, `gluLookAt`, `gluPerspective`, `gluUnProject`); NuGet package `nupengl.core`
Assets	none: no model, texture, image or sound file is loaded; all geometry is generated in code and all textures are painted at start-up

Table 2: Tools and libraries.

Technique	Source / reference followed	Ref.
Ambient, diffuse, specular light	Phong reflection model; Lambert's cosine law	[1]
Half-vector specular term	Blinn–Phong	[2]
Gouraud shading	colour interpolation across a polygon	[3]
Ray tracing (reflection, refraction)	Whitted's recursive ray tracing	[4]
Shadow map, soft edge (PCF)	Williams; Reeves, Salesin and Cook	[5], [6]
Fresnel reflection of glass	Schlick's approximation	[7]
Polygon clipping (wall pictures)	Sutherland–Hodgman	[8]
Projection, matrices, stencil, blending	OpenGL Programming Guide and GLSL 1.20 specification	[9]
Value noise, fractal Brownian motion	procedural texturing technique	[10]

Table 3: Sources followed for each technique.



Figure 1: Floor plans (the street side of the shop is at the bottom). The tatami room is covered by 8 rows of mats in a staggered pattern.

Zone	Contents
Ground floor	dining counter with 6 stools and a lacquered top, 4 ramen bowls with steam, open kitchen (two steel stock pots, stove, hood, shelves, fridge, sink), two tables with four chairs and a lamp over each, paper box lanterns, hanging lanterns, floor lantern tower, pendant glass lamps, area-light panel, noren, wall clock, shoji doors and windows
Upper floor	tatami floor with cloth borders, plaster walls with timber frame, fusuma doors, board ceiling, futon bed, chabudai tea set, cushions, tansu chest, sideboard with radio, mirror, six wall pictures, windows, ceiling and stand fans
Outdoors	cobblestone road, two flagstone sidewalks, 12 street lamps, 10 houses, cherry, maple, pine and broadleaf trees, bamboo, flowers, stone lanterns, torii gate, lake with bridge and two ducks

Table 4: What is in each zone of the scene.

Files	Responsibility
`main`, `camera`	window, frame loop, 16 ms timer, seven cameras, keyboard and mouse
`shapes`, `texture`	primitives and materials; all 13 procedural textures
`scene`, `exterior`	sky, ground, shop, upstairs room, cyclist, person, cat, chime, houses, trees, lake, props
`furniture`, `food`, `decorations`	counter, kitchen, tables, bowls and food, lanterns, noren, clock
`lighting`, `shader`	the 8 lights, day / night, Phong shader, fog, interior light scope
`shadowmap`, `refraction`	soft shadow mapping, screen-space glass refraction
`rtcore`, `rtscene`, `rtpass`	CPU ray tracer, the shop as the rays see it, threaded pass and compositing
`objects`, `game`	selectable and movable objects, the ramen cooking game

Table 5: Modules of the program.

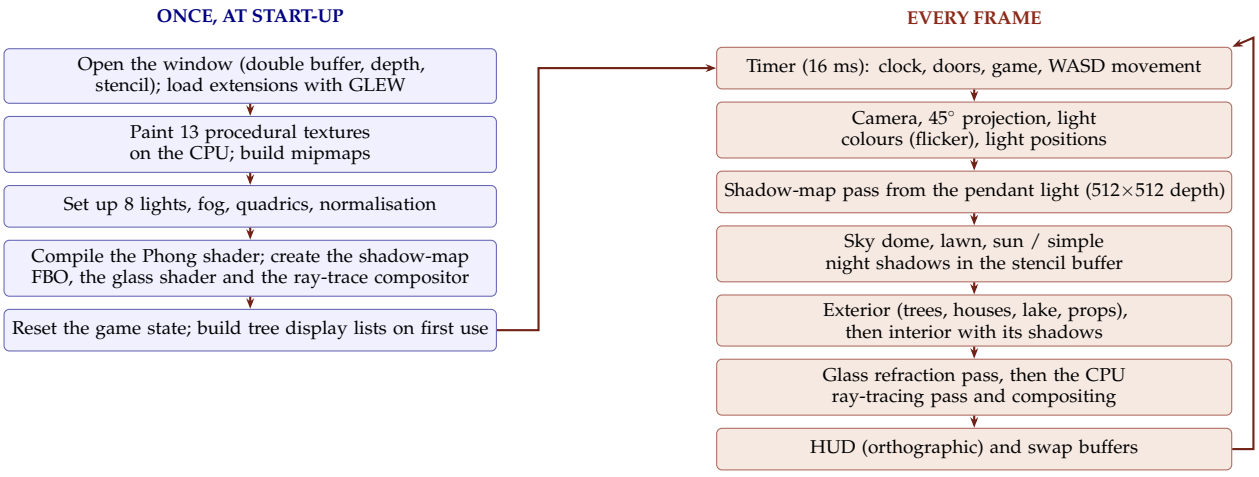


Figure 2: The rendering pipeline: work done once at start-up (left) and in every frame (right).

5.1 3D modelling from basic shapes

Theory. Complex objects are unions of simple solids. A cylinder or cone with radii r_0, r_1 and height h is a frustum. A *surface of revolution* (lathe) turns a 2D profile $(r(s), y(s))$ about the y axis:

$$\mathbf{p}(s,\theta) = \big(r(s)\cos\theta,\ y(s),\ r(s)\sin\theta\big), \qquad \mathbf{n} \propto \big(y'\cos\theta,\ -r',\ y'\sin\theta\big). \tag{1}$$

A *grid surface* samples a height function $z = f(u,v,t)$ on a mesh of quads, and a tree is a *recursive* (self-similar) structure.

Implementation. `shapes.cpp` provides `drawCube, drawCuboid, drawCylinder, drawCone, drawSphere, drawTorus, drawPlane, drawWedge` and textured boxes (Figure 3). Boxes of other sizes are cubes with non-uniform scale, ellipsoids are scaled spheres, and a tube between two points is a cylinder rotated by $\arccos(d_y/|\mathbf{d}|)$ about the axis $\mathbf{y}\times\mathbf{d}$. Bowls, lamp shades, bottles and the wind-chime bell are lathes. The nobori banner is a 12×28 grid with depth $z(u,v,t) = \big(0.09\sin(3.2t - 4u + 1.5v) + 0.015\sin(5.1t + 3v)\big)\,u$ and the normal from $\partial z/\partial u$, so the folds are shaded correctly. A tree is built by `growBranch`:

```
growBranch(base, dir, len, r, level):
    p1 = base + 0.55*len*dir;  p2 = p1 + 0.45*len*dir2        // two tapered tubes
    tube(base,p1,r,0.72r);  tube(p1,p2,0.72r,0.48r)
    if level < maxLevel:
        for k in 1..(level==1 ? 3 : 2):                       // children at a random angle
            theta = 0.45+0.45*rnd();  phi = 2*pi*rnd()
            growBranch(start_k, rotate(dir,theta,phi), 0.6*len, 0.55r, level+1)
        growBranch(p2, dir2, 0.55*len, 0.45r, level+1)        // the branch continues
    leafOrBlossomCluster(p2)                                  // leaves at the tips
```

The result is compiled into OpenGL **display lists** once (a trunk plus seven swaying branch groups, three variants and a lighter distance version, LOD), and only the sway transform is applied per frame.

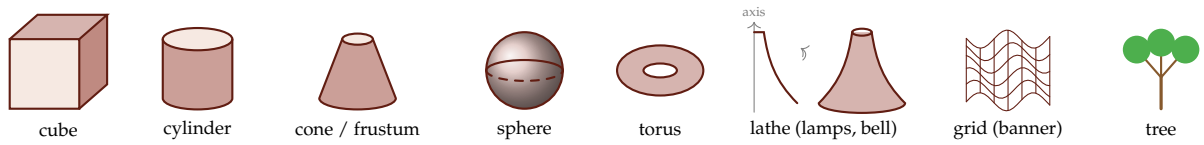


Figure 3: The construction primitives: every object in the shop is some combination of these.

5.2 Transformations and hierarchical modelling

Theory. Points are homogeneous 4-vectors, so translation, scaling and rotation are matrices:

$$T(\mathbf{p}) = \begin{pmatrix} 1 & 0 & 0 & p_x \\ 0 & 1 & 0 & p_y \\ 0 & 0 & 1 & p_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S = \operatorname{diag}(s_x, s_y, s_z, 1), \quad R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{2}$$

They are composed by multiplication, applied right to left, so $M = T\,R\,S$ scales first and moves last. A rotation about a pivot $\mathbf{c}$ is $T(\mathbf{c})R(\theta)T(-\mathbf{c})$. A *matrix stack* (`glPushMatrix`, `glPopMatrix`) lets a child part be placed in the frame of its parent, so a joint moves everything below it. Normals use the inverse transpose of the model matrix, which is why `GL_NORMALIZE` is enabled for scaled objects.

Implementation. Every object is placed by one function, `applyTransform(pos, rot, scale)` $= T\,R_x\,R_y\,R_z\,S$. Table 6 shows where each transformation is used and Figure 4 the joint hierarchies of the three articulated things.

5.3 Camera, projection and mouse picking

Theory. `gluLookAt(eye, centre, up)` builds the view matrix from the orthonormal frame $\mathbf{f} = \widehat{\mathbf{c}-\mathbf{e}}$, $\mathbf{s} = \widehat{\mathbf{f}\times\mathbf{u}}$, $\mathbf{u}' = \mathbf{s}\times\mathbf{f}$. The perspective projection matrix for vertical field of view ϕ, aspect ratio a and clip planes n, f is

$$P = \begin{pmatrix} \frac{1}{a\tan(\phi/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\phi/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \tag{3}$$

Transformation	Used for
Translation	cyclist $x(t) = x_0 + (vt \bmod L)$; person and cat walking; sliding shoji doors; rising steam; ducks on the circle $(3.5\cos 0.4t,\ 3.5\sin 0.4t)$
Rotation	wheels $\theta = d/r$ (rolling without slipping), pedals and cranks, fan blades, hinged entrance door, wind-chime pendulums, stand-fan head $35^\circ \sin 0.6t$
Scaling	cubes stretched into planks and counters; ellipsoids from spheres; cat $\times 1.8$; bicycle $\times 1.4$; user scaling of selected objects (keys , and .); breathing of the steam
Pivot rotation	$T(\mathbf{c})R(\theta)T(-\mathbf{c})$ for swaying tree branch groups, noren curtain, hanging lanterns, banner top bar
Shear / projection	flattening the scene onto the ground for shadows (Section 5.9)

Table 6: Where each 3D transformation is used.

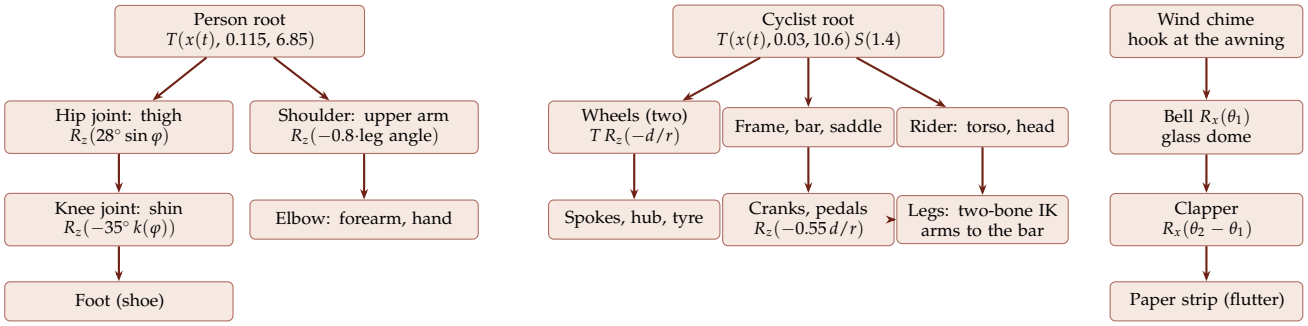


Figure 4: Parent–child transformations of the walking person, the cyclist and the wind chime. A joint moves everything below it; the dashed arrow means the rider's feet follow the pedals.

so after the division by $w = -z_e$ distant objects become smaller. An *orthographic* projection has no division by depth and keeps sizes constant, which suits a 2D overlay.

Implementation. The world uses `gluPerspective(45, aspect, 0.5, 200)` (62° in game mode); the HUD, the order board and the title screen use `glOrtho` so that the overlay keeps its pixel size. Table 7 lists the seven cameras. For **picking**, a mouse click is turned into a world ray with `gluUnProject` at depth 0 and depth 1, $\mathbf{r}(t) = \mathbf{p}_{near} + t(\mathbf{p}_{far} - \mathbf{p}_{near})$, intersected with the plane of each door, $t = (z_0 - p_z)/d_z$, and the hit point is tested against the door's rectangle.

Camera	Behaviour
Orbit (default)	orbits the shop; arrow keys, wheel and mouse drag change angle, distance and height
Walkthrough (`C`)	first person; `WASD` to move, mouse to look, stairs followed automatically
Counter, Upper Room, Pond (`C`)	orbit inside the counter area, the tatami room, and around the lake
Bowl close-up (`B`)	orbits a ramen bowl at 0.7 m: a good place to see steam, glaze and the glass ball
Light tour (`8`)	flies to the next lamp and switches it on; `9` toggles it

Table 7: The seven camera modes.

5.4 Illumination: ambient, diffuse and specular light

Theory. The light that reaches the eye from a surface point with unit normal $\mathbf{N}$, unit direction to the light $\mathbf{L}$ and unit direction to the eye $\mathbf{V}$ is the sum of three terms (Phong reflection model [1] with Blinn's half vector [2], $\mathbf{H} = \widehat{\mathbf{L} + \mathbf{V}}$; Figure 6):

$$I = \mathbf{e} + \sum_i a_i \Big[\underbrace{\mathbf{c}_a\, \mathbf{l}_{a,i}}_{\text{ambient}} + \underbrace{\mathbf{c}_d\, \mathbf{l}_{d,i}\, \max(\mathbf{N}\cdot\mathbf{L}_i, 0)}_{\text{diffuse (Lambert)}} + \underbrace{\mathbf{c}_s\, \mathbf{l}_{s,i}\, \max(\mathbf{N}\cdot\mathbf{H}_i, 0)^n}_{\text{specular}} \Big], \tag{4}$$

$$a_i = \frac{\sigma_i}{k_c + k_l\, d + k_q\, d^2}, \qquad \sigma_i = \begin{cases} 1 & \text{point light} \\ (\cos\alpha)^e \ \ (\alpha \le \text{cut-off}),\ 0 \text{ otherwise} & \text{spot light} \end{cases} \tag{5}$$

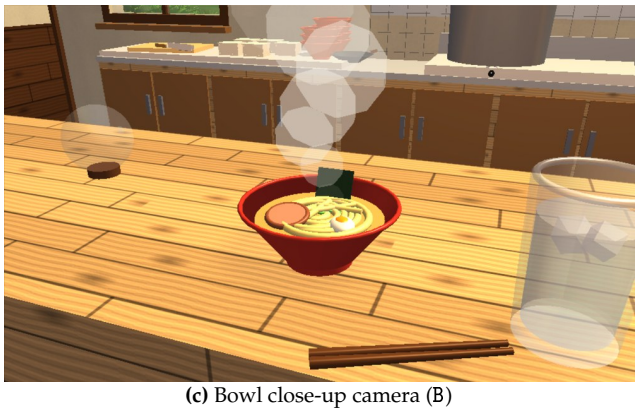
where $\mathbf{e}$ is the emission of the material (lantern paper, bulbs), $\mathbf{c}_{a,d,s}$ are the material colours, $\mathbf{l}_{a,d,s,i}$ the colours of light i, n the shininess, d the distance to the light and a_i the attenuation. A directional light (sun, moon) has $a_i = 1$. *Ambient* gives a constant glow, *diffuse* depends only on the angle between the normal and the light (matte look), and *specular* gives the bright highlight whose sharpness is n: large n is a small sharp glint, small n a broad dull one.



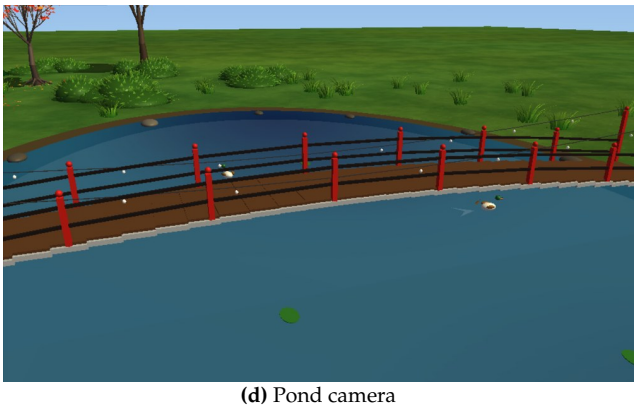
(a) Counter camera



(b) Upper-room camera



(c) Bowl close-up camera (`B`)



(d) Pond camera

Figure 5: Result: four of the seven cameras (perspective projection, 45°).

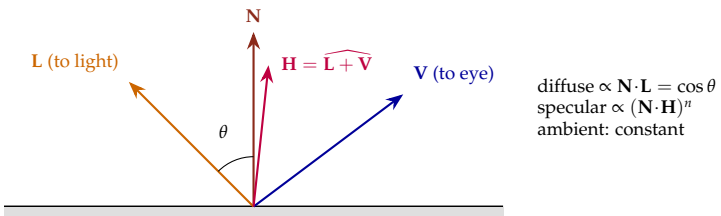


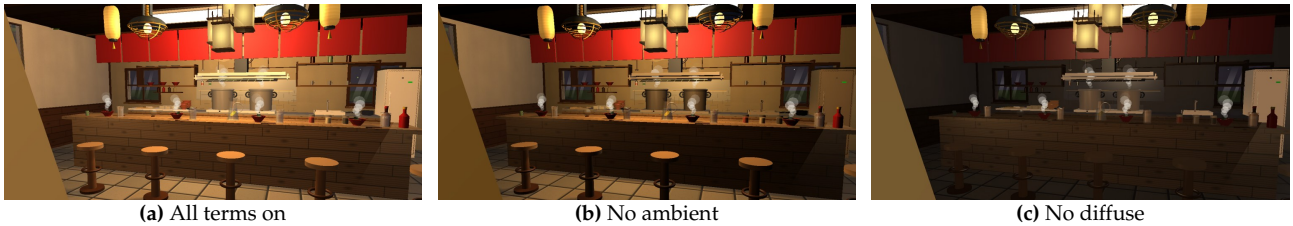
Figure 6: The vectors of the Phong / Blinn–Phong model at a surface point.

Implementation. The 8 OpenGL lights are set up in `lighting.cpp`, each with its own ambient, diffuse and specular colour (Table 8). Shininess is set per material: plaster $n = 8$, lacquered wood ≈ 55, glass 140; for ordinary dielectrics $n = 4 + 124(1-r)^2$ and the specular strength $\rho_s = 0.08 + 0.55(1-r)$ follow the roughness r, and for metals the highlight is tinted by the metal colour. The global ambient is zero, so a lamp that is switched off lights nothing. Only 8 lights exist, so the four outdoor lights (entrance lanterns, street lamps, sign) are re-assigned to interior fixtures while the inside is drawn and put back afterwards, and the shader only sees the interior set for interior objects. Keys `1-7` switch each fixture, `0` the sun or moon, and `-`, `=`, `\` remove just the ambient, diffuse or specular term from every light. Lamps also fade with a slow flicker.

Light	Type	Position	Role (outside / inside the shop)
0	directional	$(0.45, 0.70, -0.45)$	sun (warm white) or moon (blue-silver)
1	point, $k_{l,q} = 0.14, 0.07$	$(0, 2.85, 0)$	dining pendant lamps (key 1)
2	point	$(-3.8, 2.5, 4.5)$ / $(0, 2.6, 4.7)$	left entrance lantern / light through the paper wall
3	point	$(3.8, 2.5, 4.5)$ / merged	right lantern / box lanterns, hanging lanterns, ceiling panel (keys 3-5)
4	spot 28°, $e = 35$	$(-2.4, 3.25, 2.8)$	left table lamp (key 2)
5	point	$(0, 3.4, 7)$ / $(2.4, 3.25, 2.8)$	street lamps / right table lamp (key 2)
6	point	$(0, 4.5, 4.2)$ / $(-4.1, 1.4, 3.2)$	shop-sign glow / floor lantern tower (key 6)
7	point	$(0, 5.15, 0)$	upstairs paper lamp (key 7)

Table 8: The eight lights.

Result. Figure 7 switches the three terms off one at a time. Without ambient the shadows are slightly deeper; without diffuse only the glow and the highlights remain; without specular the shiny counter and pots go flat; with every light off only self-lit surfaces remain. Figure 8 shows the light tour.



(a) All terms on **(b)** No ambient **(c)** No diffuse

(d) No specular **(e)** All lights off **(f)** Gouraud shading

Figure 7: Result: the illumination terms switched off one at a time (night), and the same view with Gouraud shading.



(a) Pendant lamps (1) **(b)** Table lamp (2) **(c)** Box lanterns (4) **(d)** Lantern tower (6)

Figure 8: Result: the light tour (key 8) flies to each fixture and switches it on, showing its own glow and light pool.

5.5 Shading: Gouraud and Phong

Theory. *Flat* shading gives one colour per polygon. *Gouraud* shading [3] evaluates the illumination equation (4) only at the vertices and interpolates the colours across the triangle with barycentric coordinates, $C = \lambda_1 C_1 + \lambda_2 C_2 + \lambda_3 C_3$; a highlight that lies inside a polygon is lost. *Phong* shading interpolates the *normal*, $\mathbf{N} = \lambda_1 \mathbf{N}_1 + \widehat{\lambda_2 \mathbf{N}_2} + \lambda_3 \mathbf{N}_3$, and evaluates equation (4) at every pixel, so highlights are round and accurate. Interpolation of attributes is made perspective-correct by interpolating q/w and $1/w$ linearly in screen space.

Implementation. Key `G` switches between the two models. Gouraud is the fixed-function path (`glShadeModel(GL_SMOOTH)`). Phong is my GLSL 1.20 program in `shader.cpp`: the vertex shader passes the eye-space position, normal, colour, texture coordinate and fog distance; the fragment shader multiplies the colour by the texture, loops over the 8 lights (skipping those with `lightOn[i] = 0`), computes **L**, the attenuation, the spot cone and the three terms, then mixes the fog. Flat shading was not implemented. The core of the fragment shader:

```
vec3 N = normalize(vNormal), V = normalize(-vPos);
vec4 result = lightModelAmbient*base + material.emission;
for (int i = 0; i < 8; i++) { if (lightOn[i] < 0.5) continue;
    L = directional ? normalize(pos) : normalize(pos - vPos);   // atten = 1/(kc+kl d+kq d^2); spot: pow(cos,e)
    result += light[i].ambient * base * atten;
    float NdotL = max(dot(N,L),0.0);
    if (i==1 && shadowOn) atten *= pendantShadow();             // soft shadow of light 1
    result += light[i].diffuse * base * NdotL * atten;
    if (NdotL > 0.0) { vec3 H = normalize(L+V);
        result += light[i].specular * material.specular * pow(max(dot(N,H),0.0), shininess) * atten; } }
```

Result. In Figure 9 the same view is drawn with both models; the Phong version shows the long, smooth gloss streaks of the lacquered counter and the steel pots, which Gouraud shading loses between vertices.



(a) Phong shading (per pixel) **(b)** Gouraud shading (per vertex)

Figure 9: Result: Phong and Gouraud shading of the same view (key `G`).

5.6 Textures and texture mapping

Theory. A texture is a function $(u, v) \mapsto$ colour. *Planar mapping* of a box face uses $u = x/s$, $v = z/s$ with a world scale s, so one repeat always covers the same real length. A *mipmap* is a pyramid of pre-averaged half-size copies; the level is $\lambda = \log_2(\text{texels per pixel})$ and trilinear filtering blends two levels, which removes shimmering; anisotropic filtering keeps floors sharp at grazing angles. Procedural textures use *value noise*: random values $h(i, j)$ on an integer lattice, smoothly interpolated with $s(t) = 3t^2 - 2t^3$,

$$n(x,y) = \text{lerp}\left(\text{lerp}(h_{00}, h_{10}, s(f_x)),\ \text{lerp}(h_{01}, h_{11}, s(f_x)),\ s(f_y)\right), \qquad \text{fBm}(x,y) = \sum_{k=0}^{m-1} 2^{-(k+1)}\, n(2^k x,\ 2^k y), \quad (6)$$

and *fractal Brownian motion* (fBm) sums several octaves for natural variation at many scales. Wrapping the lattice index (`ix mod period`) makes the texture tileable.

Implementation. `texture.cpp` paints 13 textures when the program starts: wood planks, dark wood, glazed floor tile, lime plaster, asphalt, concrete, roof tile, grass, grass macro layer, water, tatami, a sphere-map environment and a soft cloud sprite. They are uploaded with mipmaps and anisotropic filtering and combined with the light by `GL_MODULATE`, $C_{out} = C_{lit} \cdot T$, so lighting still darkens and tints them. The tatami reeds use a rounded profile $k = 0.78 + 0.22\sin(\pi f) +$ row tone $+$ fibre $+$ mottling, where f is the fractional row coordinate.

Grass (my own texture)

The lawn is one 240×240 quad drawn *unlit* (so a huge polygon has no per-vertex lighting gradient) and covered by two textures.

1. *Base colour field.* Four greens $G_0 = (0.12, 0.24, 0.08)$, $G_1 = (0.22, 0.36, 0.11)$, $G_2 = (0.33, 0.46, 0.15)$, $G_3 = (0.45, 0.53, 0.21)$ are mixed by three noise octaves n_1, n_2, n_3 with the smooth step: $c = \text{mix}\left(G_1, G_2, 0.55\,\text{sstep}(0.3, 0.7, n_2)\right) \to \text{mix}\left(\cdot, G_2, 0.35\,\text{sstep}(0.45, 0.75, n_1)\right) \to \text{mix}\left(\cdot, G_3, 0.12\,\text{sstep}(0.6, 0.85, n_3)\right)$, times a $\pm 6\%$ mottling.
2. *Blade strokes.* $WH/40$ short strokes are painted, each starting at a random point, going in a direction near vertical ($90^\circ \pm 35^\circ$) 80% of the time and random otherwise, 3.5–10 texels long, with alpha 0.22 tapering to half at the tip; the colour is chosen from the greens (45% dark, 30% mid, 13% light, 7% sun-bleached, 5% dead brown).
3. *Grain.* $\pm 3\%$ per-pixel noise. The 512×512 result repeats every 14 units.
4. *Macro layer.* A second, low-contrast texture repeated every 90 units is multiplied in with the blend $C = 2\,C_{dst}C_{src}$ (neutral at 0.5), which lightens and darkens big areas and gives a slight warm / cool shift so the repeat of the detail texture cannot be seen. At night a dark green tint $(0.17, 0.22, 0.17)$ replaces white.

On top of the texture stand about 500 real 3D **grass tufts** (`buildTuft`, `drawGrassPatch`): each tuft is 30 arching blades of four quad segments, with the path $y = Ht(1 - 0.3t)$, $x = b_x + o_x\, k\, t^2 H$, the width $w = 0.03(1 -$

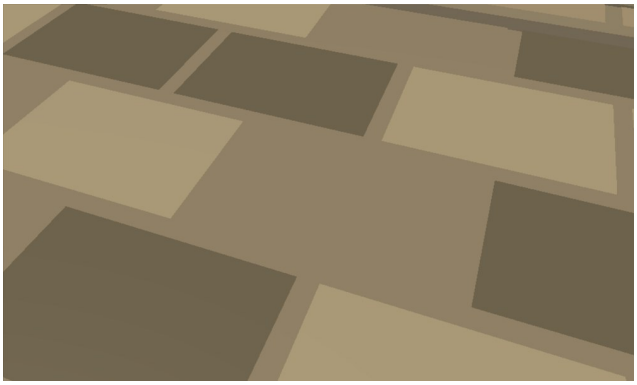
$t)^{0.8} + 0.0015$ and a colour from dark green at the root to light green at the tip. The blades sit on a sunflower spiral (golden angle 2.39996 rad), there are three variants compiled into display lists, and each tuft sways by $2.5\sin(1.5t + 0.6x + 0.4z) + 1.2\sin(3.1t + x)$ degrees; tufts farther than 32 units are skipped.

Cobblestone street and sidewalk (my own texture)

The street is built from geometry with a stone-colour hash instead of an image (`drawStreet`). A road bed 8 wide and 240 long carries stones of 1.2×0.8 separated by 0.06 grout in a brick pattern, every second row offset by half a stone. The shade of each stone comes from $h = \frac{1}{2}\sin(13.7x + 29.3z) + \frac{1}{2}$: light for $h > 0.6$, dark for $h < 0.3$, base otherwise. Raised curbs and a darker centre line finish it. The sidewalks (`drawSidewalk`) use 1.4×1.4 flagstones in two tones chosen by $\frac{1}{2}\sin(17.3x + 41.7z) + \frac{1}{2}$ and a low stone border.



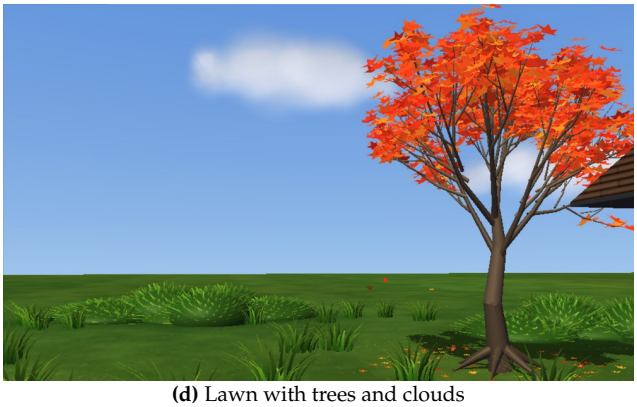
(a) Grass: texture, 3D tufts, flowers



(b) Cobblestone street



(c) Tufts on the lawn beside the lake

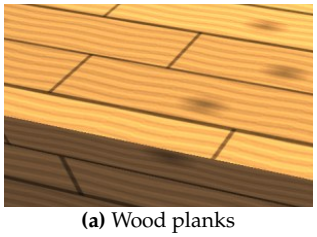


(d) Lawn with trees and clouds

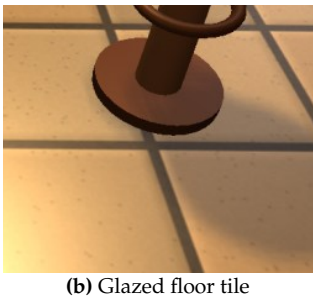
Figure 10: Result: the grass and street textures I made for the outdoors.

Other textures

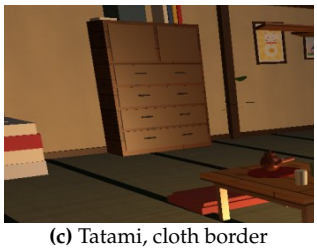
Wood planks give the counter and floors; glazed tile the dining floor; lime plaster the walls; corrugated tile the roofs of the shop and houses; woven tatami the upstairs floor; and a smooth blue gradient with soft ripples ($\sin(8\,\text{dist})$ wave and a swell term) the lake, whose coordinates scroll over time by $0.04 \cdot 0.8t$. The sphere-mapped environment texture (`GL_SPHERE_MAP`) gives the pots and glass a faint reflection of the room.



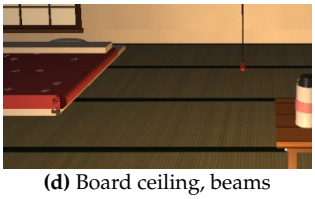
(a) Wood planks



(b) Glazed floor tile



(c) Tatami, cloth border



(d) Board ceiling, beams

Figure 11: Result: procedural textures as they appear in the shop.

5.7 Skybox, sun, moon, stars, clouds and day / night

Theory. A *skybox* is drawn first, with the depth test and depth writes off and with the camera translation removed from the modelview matrix (only the rotation kept), so it appears infinitely far away and the rest of the

scene is drawn over it. Glowing things add light, so halos use *additive blending* $C = \alpha C_s + C_d$, while ordinary transparency uses $C = \alpha C_s + (1-\alpha)C_d$. A *billboard* is a quad rotated to always face the camera.

Implementation. `drawSky()` paints four layers (Table 9).

Layer	Construction
Dome	14 rings of 36 triangle strips, radius 150, elevation $-10°$ to $90°$; the vertex colour is $\text{mix}(\text{horizon}, \text{zenith}, t)$ with $t = (e/90°)^{0.55}$. Day $(0.64, 0.79, 0.93) \to (0.17, 0.40, 0.80)$; night $(0.10, 0.10, 0.16) \to (0.015, 0.02, 0.07)$. By day the horizon equals the fog colour, so far ground melts into the sky.
Sun	five camera-facing triangle-fan discs at direction $(45, 7, -63)$: halos of radius 26, 13 and 7 (alpha 0.12/0.22/0.35 fading to 0 at the rim, additive), a core of radius 3 and a bloom of radius 4; warm yellow to white
Moon	the same position at night: cool halos (24, 11, 6), a disc of radius 2.8 and six darker craters (fans of radius 0.3–0.7, alpha $0.55 \to 0$)
Stars	132 points (57 bright of 3 px, 75 dim of 1.5 px); brightness $0.7 + 0.3\sin(\omega_i t + \varphi_i)$ with its own frequency and phase
Clouds	12 clouds of 22 billboards each, one 128×128 RGBA noise sprite, a flat base and a domed top, sorted low to high, drifting with azimuth $a_0 + vt$

Table 9: Layers of the sky.

```
glDisable(GL_DEPTH_TEST);  glDepthMask(GL_FALSE);  glDisable(GL_FOG);
glGetFloatv(GL_MODELVIEW_MATRIX, mv);  mv[12] = mv[13] = mv[14] = 0;  glLoadMatrixf(mv);  // rotation only
drawGradientDome();  drawSunOrMoon();  if (night) drawStars();  drawCloudBillboards();
glDepthMask(GL_TRUE);  glEnable(GL_DEPTH_TEST);                                          // scene on top
```

Key `T` switches day and night. It changes the sky colours, the colour of the directional light (warm gold $(0.85, 0.78, 0.55)$ by day, cool blue $(0.14, 0.16, 0.25)$ at night), the glow of every lantern and window (about 15–20% by day), the street lamps, the fog density ($\rho = 0.012$ or 0.022), the fireflies and the sun shadows. **Result.**

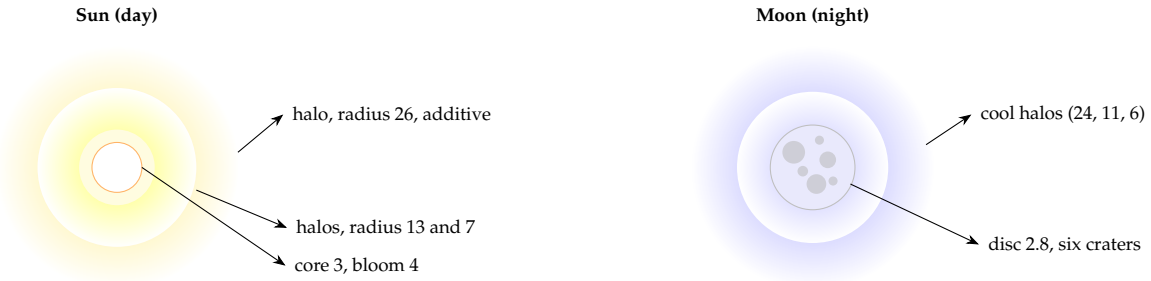


Figure 12: The sun and the moon are stacks of transparent discs. Light adds up, so the halos use additive blending and the core uses alpha blending.

Figure 13 shows the sun's glow behind the trees by day, a cloud, the cratered moon with fireflies and a street lamp, and the twinkling stars with dark clouds at night.

5.8 Blending, transparency, emission and fog

Theory. Alpha blending mixes a source colour with the frame buffer, $C = \alpha C_s + (1-\alpha)C_d$. The depth buffer cannot sort translucent surfaces, so they are drawn after all opaque ones with depth *writes* off. Fog moves the colour toward a fog colour with the exponential-squared factor

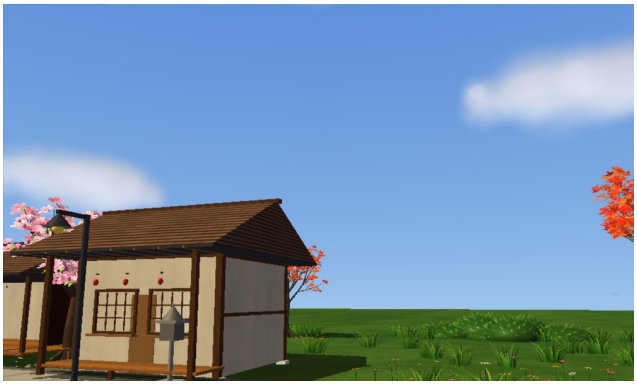
$$f = e^{-(\rho z)^2}, \qquad C = f\, C_{surface} + (1-f)\, C_{fog}, \tag{7}$$

where z is the eye distance and ρ the density.

Implementation. Glass, water, steam, clouds and soft shadows use alpha blending; halos, glows, light cones and fireflies use additive blending. All glass (windows, jars, tumblers, the pitcher, pendant shades) is drawn in a final transparent pass (`drawTransparentPass`) so the scenery behind it is not painted over it. Steam is seven translucent spheres per bowl with phase $\varphi = (0.65t + \text{offset} + i/7) \bmod 1$, radius $(0.09 + 0.32\varphi)$, height 1.5φ and alpha $0.28\sin(\pi\varphi)$, drifting sideways. Fog is applied at the end of the Phong shader and by `glFog` in the fixed path, with $\rho = 0.012$ by day and 0.022 at night.



(a) Day: sun glow near the horizon, blue sky



(b) Day: cloud billboards



(c) Night: the cratered moon, fireflies, lamp



(d) Night: stars and dark clouds

Figure 13: Result: the sky by day and at night.

5.9 Shadows

Theory. *Planar projective shadow.* A point is flattened onto the plane $y = h$ along the light direction $\mathbf{s}$ (pointing toward the light):

$$x' = x - \frac{s_x}{s_y}(y - h), \qquad y' = h, \qquad z' = z - \frac{s_z}{s_y}(y - h), \tag{8}$$

which is a single 4×4 matrix; for a point light at $\mathbf{l}$ the same is $x' = \dfrac{l_y\, x - l_x\, y}{l_y - y}$. *Shadow mapping* (Williams [5]) renders the scene from the light into a depth texture; a point whose depth in light space is z_L is in shadow when $z_L > z_{map} + \epsilon$ (ϵ is a bias against *shadow acne*). *Percentage-closer filtering* [6] averages several such comparisons to get a soft edge,

$$s = \frac{1}{N} \sum_{k=1}^{N} \left[z_L \le z_{map,k} + \epsilon \right]. \tag{9}$$

Implementation. Three kinds of shadow are used.

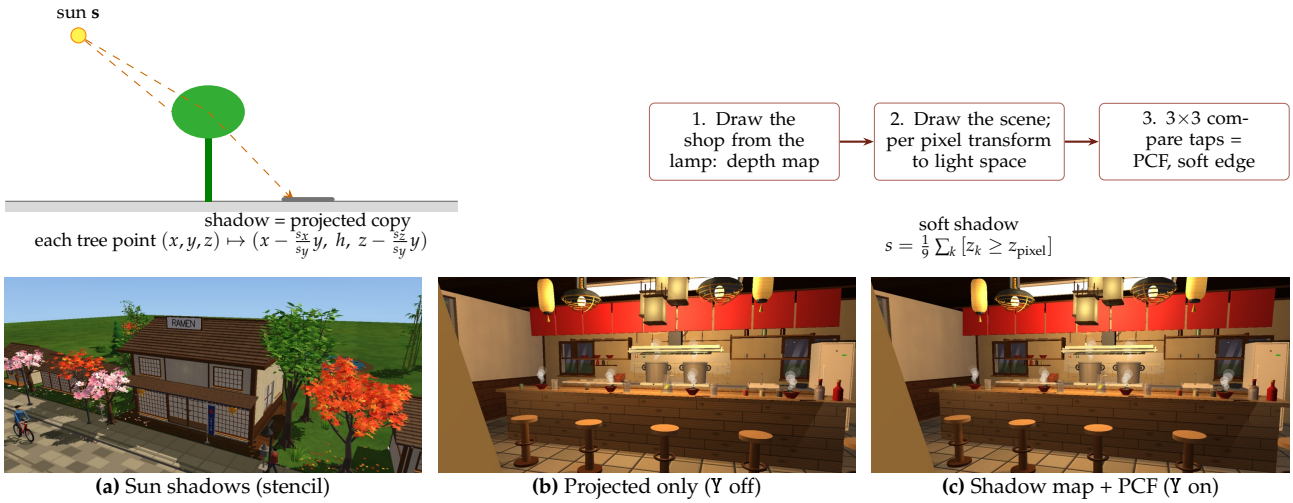
1. **Sun shadows (outdoors).** The whole outdoor scene is drawn a second time, flattened with the matrix above, into the **stencil buffer only**; a single soft dark layer is then blended where the stencil is set, so overlapping branches never darken twice and every shadow has exactly the outline of its object. Trees near the shop use their real leaf geometry, far trees cheap solid crowns. At night a few simple fixed soft shadows are drawn instead.
2. **Interior shadows.** The furniture is projected onto the tiled floor from the pendant and table lamps with the point-light version of the matrix and a stencil counter ($1 \to 2 \to 1$).
3. **Soft shadows (key `Y`).** The counter, stools, tables, chairs and bowls are drawn from the dining pendant light into a 512×512 depth texture (FOV 120°, `glPolygonOffset` against acne). In the fragment shader every pixel is transformed into the light's clip space and compared 3×3 times, 1.5 texels apart (each already a bilinear 2×2 hardware compare), and averaged (`pendantShadow()` in Section 5.5).

The stencil pass of the sun shadow (`shadowLayer` in `scene.cpp`, shortened):

```
glDisable(GL_DEPTH_TEST);  glEnable(GL_STENCIL_TEST);
glStencilFunc(GL_EQUAL, 0, 0xFF);              // exterior ground only (stencil 0)
glStencilOp(GL_KEEP, GL_KEEP, GL_INVERT);      // first fragment flips 0 -> 0xFF, later ones fail
glColorMask(0,0,0,0);  glDepthMask(GL_FALSE);  // write only the stencil
glPushMatrix();  multShadowMatrix(mode, lx, ly, lz, groundY);  drawExteriorBody();  glPopMatrix();
```

```
glColorMask(1,1,1,1);
glStencilFunc(GL_EQUAL, 0xFF, 0xFF);  glStencilOp(GL_KEEP, GL_KEEP, GL_ZERO);   // blend once, clear mask
glEnable(GL_BLEND);  glColor4f(0.01, 0.03, 0.01, alpha);  shadowRegion(sidewalk, groundY);   // dark layer
```

Depth testing is turned off in this pass because the shadow plane lies a few millimetres above the paving and would otherwise z-fight.



(a) Sun shadows (stencil) **(b)** Projected only (Y off) **(c)** Shadow map + PCF (Y on)

Figure 14: Result: how a tree is flattened onto the ground and the steps of soft shadow mapping (top); the three kinds of shadow in the scene (bottom).

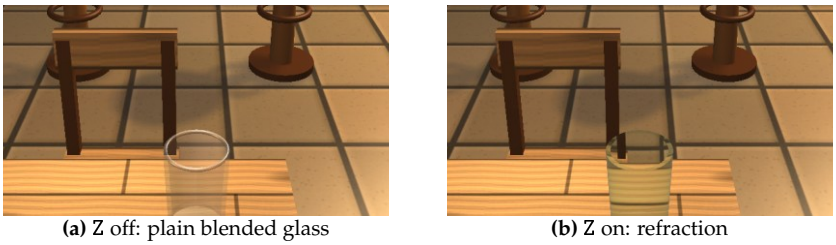
5.10 Glass refraction (screen space)

Theory. Light bends when it enters glass. Snell's law $n_1 \sin\theta_1 = n_2 \sin\theta_2$ gives the bent direction, with $\eta = n_1/n_2$ and $c = -\mathbf{n}\cdot\mathbf{d}$,

$$\mathbf{t} = \eta\,\mathbf{d} + \left(\eta\,c - \sqrt{1-\eta^2(1-c^2)}\right)\mathbf{n}. \tag{10}$$

The share of light that is reflected instead is given by Schlick's approximation [7], $F = F_0 + (1-F_0)(1-\cos\theta)^5$ with $F_0 = \left(\frac{n_1-n_2}{n_1+n_2}\right)^2 \approx 0.04$ for glass ($n = 1.5$).

Implementation. The empty tumbler on the left table is drawn last, over the finished picture. The screen area behind it is copied into a texture and the tumbler is drawn with a GLSL shader that shifts the texture lookup by the amount `refract(I, N, 1/1.5)` bends the view ray at the glass normal, so the table and floor behind it appear displaced and distorted. A Fresnel reflection F adds the reflected warm room at grazing angles and the lamps leave sharp highlights ($n = 140$). It is an approximation: only what is already on the screen can be seen through the glass (key Z).



(a) Z off: plain blended glass **(b)** Z on: refraction

Figure 15: Result: the tumbler without and with screen-space refraction; the floor and table edge behind it are bent.

5.11 Ray tracing

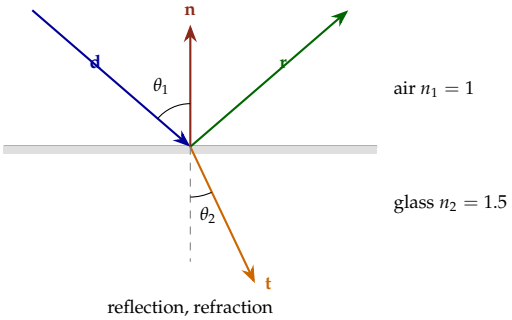
Theory. Rasterisation asks, for each triangle, which pixels it covers. Ray tracing asks, for each pixel, what its ray hits. A ray is $\mathbf{r}(t) = \mathbf{o} + t\,\mathbf{d}$ with $t > 0$. At the nearest hit (point $\mathbf{p}$, unit normal $\mathbf{n}$) the colour is the local Blinn–Phong colour plus recursively traced rays (Whitted [4]):

- a **reflection** ray with direction $\mathbf{r} = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\,\mathbf{n}$;
- a **refraction** ray $\mathbf{t}$ from Snell's law (Section 5.10); if $1-\eta^2(1-c^2) < 0$ the ray is totally internally reflected;
- the two are weighted by the Fresnel term, $C = (1-F)\,C_{local \text{ or } refr} + F\,C_{refl}$.

The recursion stops at a fixed depth (3 here). To find the hit, each ray is intersected with the shapes:

- **sphere** $|\mathbf{p}-\mathbf{c}|^2 = R^2$: substituting the ray gives $|\mathbf{d}|^2t^2 + 2\,\mathbf{d}\cdot(\mathbf{o}-\mathbf{c})\,t + |\mathbf{o}-\mathbf{c}|^2 - R^2 = 0$; a negative discriminant is a miss;
- **plane / disk**: $t = \dfrac{(\mathbf{q}-\mathbf{o})\cdot\mathbf{n}}{\mathbf{d}\cdot\mathbf{n}}$, then the hit must lie between the inner and outer radius;
- **axis-aligned box** (slab method): per axis $t_{1,2} = (b_{min,max} - o)/d$; the ray hits if $\max t_{near} \le \min t_{far}$;
- **cone / frustum** (pots, bowls, glass walls): a quadratic in t whose root must lie in the height range of the cone.

A **bounding volume hierarchy** (BVH) wraps groups of shapes in boxes; a ray that misses a box skips everything inside it, so the cost per ray falls from $O(n)$ to about $O(\log n)$ (Figure 16).



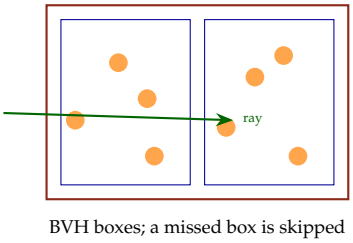


Figure 16: Left: the reflected ray **r** and refracted ray **t** at a surface. Right: a bounding volume hierarchy.

Implementation. The ray tracer (`rtcore.cpp`, `rtscene.cpp`, `rtpass.cpp`) is switched on with key `R`.

- **What is traced.** Eight objects: the 2 steel stock pots, the 4 ramen bowls, the glass ball on the counter and the mirror upstairs. The rest of the frame stays OpenGL; the tracer only decides the pixels of these objects.
- **Scene.** `rtBuildShopScene` rebuilds the shop for the rays from the same positions and sizes as the OpenGL models (boxes, truncated cones, disks and spheres), with CPU copies of the textures and the live OpenGL lights. Lamps are emissive shapes, so they appear in reflections. The BVH uses a median split on the longest axis with at most 2 shapes per leaf; large floors and walls are tested directly.
- **Shading.** Metal, glaze, soup and the mirror add a reflection ray weighted by Schlick's F (up to `maxDepth` $= 3$); the glass ball applies Snell's law at every surface it crosses, with total internal reflection.
- **Speed.** Only the screen rectangle of each traced object is processed, the rows are shared by up to 8 CPU threads, and a ray budget keeps the pass near 25 ms by tracing one ray per 2×2 (or larger) block when a big area is on screen. Measured: about 13 ms in a Release build (about 7 000 eye rays on 8 threads) and about 80 ms in Debug.
- **Compositing.** A shader draws the result over the OpenGL frame only where the OpenGL depth buffer shows the same surface, so a nearer object still hides a pot.

The recursion of the tracer (`radiance` and `glassRadiance` in `rtcore.cpp`, simplified):

```
radiance(o, d, depth):
    hit = bvhIntersect(o, d)                      // nearest shape; big floors/walls tested directly
    if no hit: return skyColour
    N = normal facing the ray;  V = -d
    EMISSIVE: return emission                     // lamps: they show up in reflections
    DIFFUSE : return blinnPhong(hit, N, V)        // all lights, no shadow rays
    GLASS   : return glassRadiance(hit, d, depth)
    METAL / GLAZE / LIQUID / MIRROR:
        F = schlick(f0, dot(N,V));  local = blinnPhong(hit, N, V)
        if depth >= maxDepth: return local
        return local*(1-F) + radiance(p+N*eps, reflect(d,N), depth+1) * F

glassRadiance(hit, d, depth):                     // follows the ray through up to 10 surfaces
    repeat:  entering = dot(d,n) < 0;  eta = entering ? 1/ior : ior
             t = refract(d, n, eta)               // fails => total internal reflection
             F = tir ? 1 : R0 + (1-R0)*(1-cos)^5
             if first surface: colour += radiance(p, reflect(d,n), depth+1) * F
             if tir: d = reflect(d,n)  else: { throughput *= (1-F);  d = t }
             next = next surface of the same glass object, else the whole scene
             if next is not glass: return colour + throughput * radiance(p, d, depth+1)
```

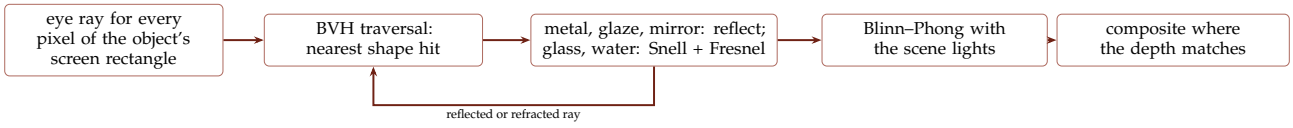


Figure 17: The steps taken for every eye ray of a ray-traced object.

Result. Figure 18 compares the same kitchen view with ray tracing off and on: the steel pots mirror the shop. Figure 19 shows a ramen bowl and the glass ball on the counter.



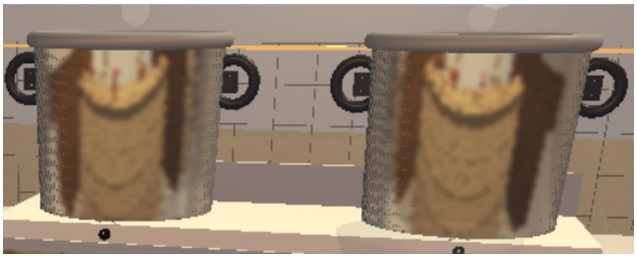
(a) OpenGL only (R off)



(b) Ray tracing on (R)

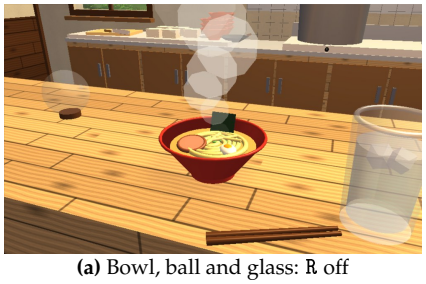


(c) Pots, OpenGL (zoom)



(d) Pots, ray traced: reflection of the shop

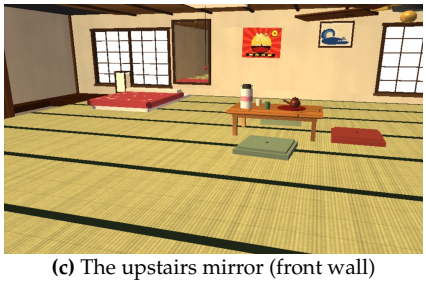
Figure 18: Result: ray-traced reflection on the steel stock pots. Up close one ray covers several pixels, so the reflection looks slightly blocky.



(a) Bowl, ball and glass: R off



(b) R on: the glass ball refracts the counter



(c) The upstairs mirror (front wall)

Figure 19: Result: the glass ball (reflection and refraction with Snell's law) and the mirror.

5.12 Animation and the characters

Theory. Every motion is a function of one clock t (`animTime`, advanced by a 16 ms timer; `Space` pauses it). A wheel that has rolled a distance d has turned $\theta = d/r$. A chain of joints is *forward kinematics* (a stack of rotations); *inverse kinematics* finds the joint from the target. For a two-bone limb with hip $\mathbf{h}$, foot $\mathbf{f}$, bone lengths L_1, L_2 and $d = |\mathbf{f} - \mathbf{h}|$ (clamped to $L_1 + L_2$),

$$a = \frac{L_1^2 - L_2^2 + d^2}{2d}, \qquad h = \sqrt{L_1^2 - a^2}, \qquad \mathbf{k} = \mathbf{h} + a\,\hat{\mathbf{u}} + h\,\hat{\mathbf{u}}^{\perp}, \tag{11}$$

so the knee always bends forward and the shin reaches the pedal exactly. A gentle start and end of a motion uses the smoothstep $3k^2 - 2k^3$. A pendulum driven by wind obeys $\ddot{\theta} = -\frac{g}{L}\sin\theta - c\dot{\theta} + k\,w(t)$, integrated by semi-implicit Euler steps ($\dot{\theta} \leftarrow \dot{\theta} + \ddot{\theta}\,\Delta t$, then $\theta \leftarrow \theta + \dot{\theta}\,\Delta t$), which stays stable.

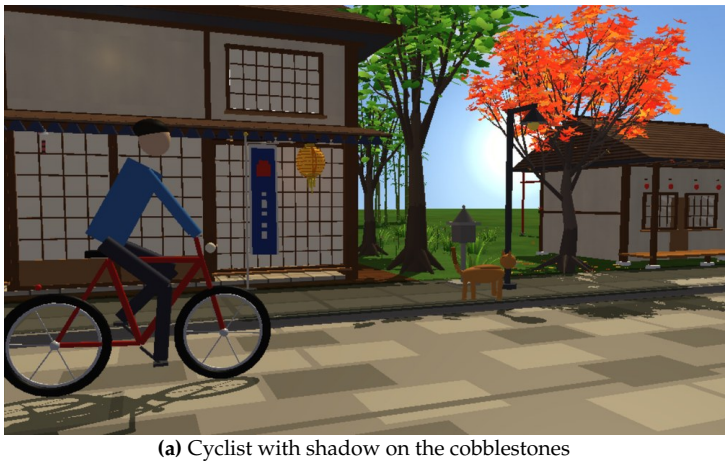
Implementation. (`scene.cpp`). A helper `limb(a, b, r)` draws a cylinder between two points; all characters use only spheres, cylinders and boxes.

- **Cyclist** (`drawBicycle`). The rider crosses the street from $x = -16$ to 16 at 7 units/s and starts again, scale 1.4. Each wheel is a tyre (torus), a steel rim, six spokes and a hub, turned by $\theta = -d/(rS)$. Two cranks of length 0.17 turn at $0.55\,d/(rS)$, 180° apart, and the legs ($L_1 = L_2 = 0.47$) reach the pedals by the IK above. The frame is seven red tubes; at night a headlight sphere glows and an additive beam lights the road.

```
dist  = fmod(animTime*7, 32);   x = -16 + dist;          // 7 units/s, loops every 32 units
wheelDeg = -(dist / (R*SC)) * 57.29578;                  // rolling without slipping
```

```
crank    = -(dist / (R*SC)) * 0.55;                          // pedals turn slower than wheels
for each pedal p:  P[p] = bottomBracket + 0.17*(cos(a), sin(a));  a = crank + p*PI
    knee = kneeFor(hip, P[p], 0.47, 0.47);                  // two-bone IK (law of cosines)
    limb(hip, knee, 0.055);  limb(knee, P[p], 0.045);       // thigh, shin
```

- **Walking person** (`drawWalkingPerson`). A man in a red jacket walks along the sidewalk at 1.8 units/s, $x = 13 - ((1.8t + 4) \bmod 26)$, as a joint hierarchy: hip → thigh (0.46) → knee → shin (0.44) → shoe, and shoulder → upper arm (0.30) → elbow → forearm (0.27) → hand. With the stride phase $\varphi = 2\pi \cdot 1.8t/1.4$: thigh angle $\pm 28^\circ \sin\varphi$ (the two legs half a period apart), knee bend $35^\circ \max\big(0, \sin(\varphi + \varphi_0)\big)$ so the knee bends only while the leg swings forward, arm swing $-0.8\times$ the leg angle with a relaxed 15° elbow, and a body bob of $0.035|\cos\varphi|$.
- **Cat** (`drawWalkingCat`). An orange tabby walks at 3 units/s. Every *third lap* a time-driven **state machine** stops it at $x = -1.2$ in front of the shop, turns it to the street (smoothstep over 0.5 s), lowers it into a sitting pose and lets it wash its face for 7 s: the right paw rises to the face, licks ($\sin 9t_w$), wipes over the ear ($0.5 + 0.5\sin 2.2t_w$) while the head tilts by $-14^\circ + 10^\circ$·wipe, then it looks around ($25^\circ \sin 0.8t$) and walks on. Walking uses diagonal leg pairs: swing $0.06\sin(\varphi + \varphi_i)$, lift $0.025\max(0, \cos(\varphi + \varphi_i))$, body bob $0.012|\sin\varphi|$ and a five-segment tail that sways.
- **Wind chime** (`drawFurin`). The bell ($L_1 = 0.12$) and the paper strip ($L_2 = 0.20$) are two coupled pendulums with wind gains $k_1 = 3$ and $k_2 = 22$ (the paper catches much more wind), 8 sub-steps per frame. When $|\theta_2 - \theta_1| > 0.42$ rad the clapper strikes the glass, rebounds with -0.45 of the relative speed and a star-shaped glint flashes.
- **Other motion.** Nobori banner (grid wave of Section 5.1), ceiling fan 160°/s, stand fan 900°/s with its head swinging $35^\circ \sin 0.6t$, swaying lanterns $\pm 2.5^\circ$ with a $\pm 8\%$ flicker, steam, ducks, falling petals, fireflies; entrance door hinge 2.5°/frame and sliding doors 2.5%/frame, opened by a click.



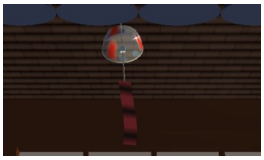
(a) Cyclist with shadow on the cobblestones



(b) Person and cat on the sidewalk



(c) Cyclist, person and cat in the street



(d) Wind chime



(e) Porch: banner, chime

Figure 20: Result: the animated characters, the wind chime and the banner.

5.13 Environment: trees, village, lake and the shop

5.14 Interaction and the ramen cooking game

Interaction. Keyboard and mouse events arrive through GLUT callbacks and change global state that the next frame reads. `C` cycles the camera, `T` day / night, `1-7` lamps, `G` Gouraud / Phong, `Y` soft shadows, `Z` refraction, `R` ray tracing, `H` roof, `O` outlines, `F` fog, `X` steam, `Space` pause. `Tab` selects a ramen bowl, pot, lantern, noren curtain or stool, which can then be moved (`I J K L U N`), rotated (`[ ]`), tilted (`; '`) and scaled (`, .`) by the delta-transform system. A mouse click opens or closes the doors (ray–plane picking, Section 5.3).



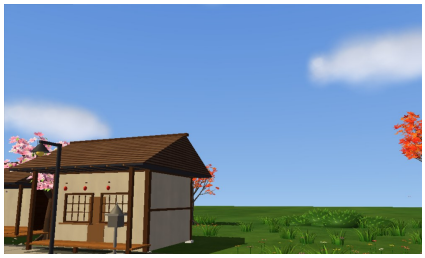
(a) Cherry blossom tree



(b) Cherry and maple trees by the shop



(c) Pine and maple



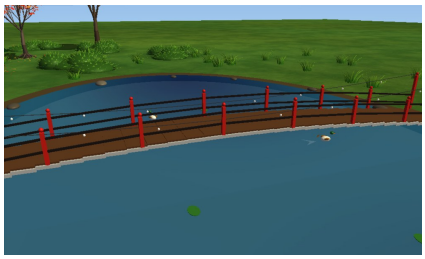
(d) Neighbouring house, type 1



(e) Neighbouring house, type 2



(f) Street lamp and lantern at night



(g) Lake, bridge and lily pads



(h) Duck and wake



(i) Shop, awning and roof

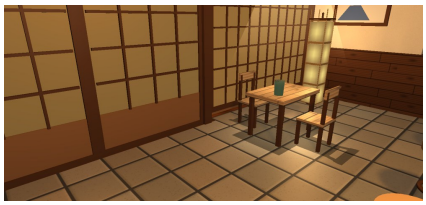
Figure 21: Result: trees, houses, street lighting, lake and the shop front.



(a) Counter, stools, pots, lanterns



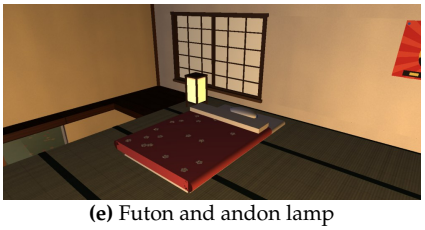
(b) Kitchen and counter top



(c) Customer table



(d) Dining room and window light



(e) Futon and andon lamp



(f) Pictures drawn in code

Figure 22: Result: the inside of the shop and the tatami room.

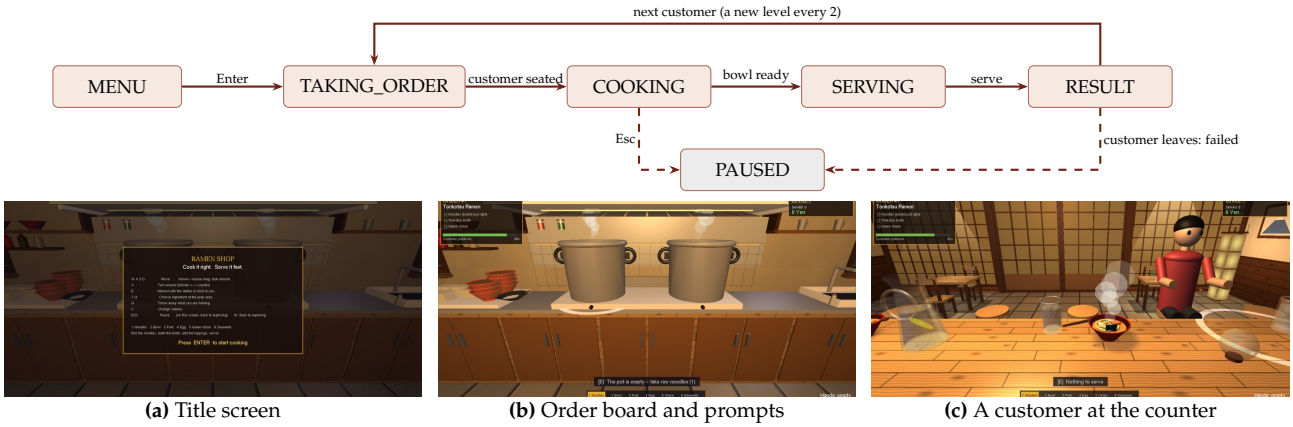
Object (function)	Count	How it is made
Cherry tree (`drawCherryBlossomTree`)	7	3-level recursion, pink blossom clusters, 14 falling petals and a carpet of 110 petals on the ground
Maple (`drawMapleTree`)	6	4-level finer branching, red-orange notched maple leaves, falling leaves
Broadleaf (`drawTree`)	6	3 levels, leaf pads on tips and mid-branch, flared trunk with five roots
Pine (`drawJapanesePineTree`)	5	trunk 3.3 tall and stacked jagged triangle-fan skirts (18 points, light centre, dark drooping rim); the crown sways
Bamboo (`drawBambooGrove`)	2	culms 4.6–6.6 tall with 9–11 nodes, a random lean and an age-dependent green–yellow colour, leaves at the top
Jungle, flower gardens	8, 9	dome bushes and fern fronds; 40 flowers per garden on a golden-angle spiral of radius 3 in seven colours, 6–8 petals around a yellow centre
Lake, bridge, ducks	1, 1, 2	disc of radius 8, scrolling water texture, rippling rim $0.015\sin(0.8t + 3a)$, muddy bank, 12 rocks, 4 lily pads; arched bridge of 34 planks on $y = 0.6\cos(\pi z/17)$ with red posts; ducks on circles of radius 3.5 with a wake
Fireflies	129	8 groups; slow orbits, pulse $0.5 + 0.5\sin(\omega t + \varphi)$ and on/off phase, additive, night only
Street lamps, stone lanterns	12, 4	pole 3.3 tall, arm and cone shade; emissive bulb, additive light cone and ground pool at night; stone base, pillar, glowing chamber, pagoda cap
Houses (`drawJapaneseHouse`, `...House2`)	10	type 1: 4.0×3.5, veranda, four red lanterns; type 2: 6.0×3.5 on a raised floor, four sliding shoji panels, side porch; textured walls and tiled roofs with exposed rafters; warm light spill at night
Shop (`drawShopBuilding`, `drawRoof`)	1	10×8 floor, plaster walls with openings, gable roof $11.4 \times 2.2 \times 9.4$, awning edged by 37 blue cones, lit "RAMEN" sign, shoji windows that glow only when an interior lamp is on, two swaying lanterns, doors that open by click
Dining room, kitchen	–	counter, 6 stools, 2 tables, 4 chairs, 4 bowls with steam, 2 stock pots, stove, hood, fridge, sink, pendant lamps, lanterns, noren, clock, glassware
Upstairs room	–	staggered tatami, fusuma and shoji doors, futon, tea table with teapot, tansu chest with drawers, radio, fans, mirror, six pictures painted from 2D shapes and clipped by Sutherland–Hodgman [8]

Table 10: The objects of the scene and how they are made.

Game. The game (`game.cpp`, opened with `Enter`) is a state machine, Figure 23. A customer walks in, sits and orders; the player takes raw noodles (1) to the noodle pot, takes a bowl (2), scoops the noodles when they look perfectly boiled (the colour changes with the boiling time: raw < 3 s, perfect 6–9 s, ruined > 12 s), ladles broth, adds toppings (3–6) and serves. The bowl is graded

$$Q = 0.5\,\text{accuracy} + 0.35\,\text{cook} + 0.15\,\text{time}, \qquad \text{stars} = \text{round}(5Q), \tag{12}$$

where *cook* is $0.15, 0.60, 1.00, 0.60, 0.10$ for raw, undercooked, perfect, overcooked and ruined, and *time* is the patience left (from $105 - 14(\text{level} - 1)$ s, at least 50 s). Money is $\lfloor 10\,Q\,(1 + 0.1(\text{level} - 1)) + 0.5\,\text{stars} \rfloor \times 10$ yen, plus 20 for a five-star bowl; a new level (up to 4) starts every two customers. The HUD is drawn with the orthographic projection.



(a) Title screen **(b)** Order board and prompts **(c)** A customer at the counter

Figure 23: Result: the state machine of the game and three of its screens.

6 Results

The scene runs in real time with all effects switched on. By day the sun and sky fill the scene and trees cast shadows on the lawn, sidewalk and street; at night the street lamps and lanterns glow, the shop windows shine through the paper and fireflies drift over the grass; inside, the lacquered counter and the steel pots catch bright highlights, the lamps warm the wood and the stools throw soft shadows.



(a) Day **(b)** Night

Figure 24: Result: the shop from the street by day and at night.

Item	Value	Item	Value
Lights / switchable fixtures	8 / 7	Camera modes	7
Procedural textures	13	Houses, street lamps	10, 12
Trees (cherry, maple, broadleaf, pine)	7, 6, 6, 5	Stars, cloud billboards	132, 12
Shadow map	512 × 512, PCF 3×3	Ray-traced objects	8
Ray-trace cost (Release / Debug)	13 ms / 80 ms	Moving characters	3 + 2 ducks
Source files, lines of code	35, about 15 000	External assets	none

Table 11: Scene statistics. The ray-trace time was measured in a close view with 6–7 featured objects (about 7 000 eye rays) on 8 threads.

Parameter	File	What changing it does
`SHADOW_MAP_SIZE` (512)	`shadowmap.cpp`	1024 gives sharper soft shadows at a higher GPU cost
Spot cut-off 28°, exponent 35	`lighting.cpp`	wider cone / softer centre of the table lamps
k_l, k_q of each light	`lighting.cpp`	how far each lamp reaches (pool of light)
`TARGET_MS` (25)	`rtpass.cpp`	smaller: smoother frame rate but blockier ray tracing
`TREE_LOD_DIST` (18)	`exterior.cpp`	distance at which trees switch to the lighter build
Wind-chime gains k_1, k_2	`scene.cpp`	how strongly the wind pushes the bell and the strip
Cyclist, person, cat speeds	`scene.cpp`	how fast they move along the street

Table 12: Parameters that can be changed to see their effect.

7 Problems, Limitations and Future Work

What worked. Building every object from a few primitives kept the code small and the scene consistent, and the per-light switches made the lighting model easy to inspect. The stencil trick gives clean, non-overlapping tree shadows cheaply, and the hybrid of rasterisation with a small ray tracer shows real reflection and refraction without tracing the whole scene.

Problems and how they were solved.

- *Only 8 lights.* The outdoor lights are re-assigned to interior fixtures while the inside is drawn, and a shader mask keeps outside lights from lighting the interior through the walls.
- *Lights switched off but the room still lit.* A global ambient and glowing materials caused it; the global ambient is now zero and glowing surfaces fade with their lamp, so every term belongs to a light.
- *Broken, streaky shadows.* The shadow planes were a few millimetres above the ground and z-fought, and leaf cards left gaps in the stencil; depth testing is off for that pass and trees use full leaf geometry near the shop.
- *Blocky ray-traced objects up close.* The ray budget keeps the cost bounded, so a large on-screen object gets one ray per several pixels; the drinking glasses and the jug, which looked worst, are therefore drawn by OpenGL, and the table tumbler keeps its screen-space refraction.

Limitations. Flat shading was not implemented; refraction of the tumbler is a screen-space approximation; ray tracing covers only eight interior objects and uses no shadow rays; the outdoor night shadows are simple fixed shapes; there is no sound. **Future work:** cube-map shadow maps for every lamp, a shadow map for the sun, ray tracing in a compute shader with a GPU BVH, and customers that walk into the shop by themselves.

8 Conclusion

The Ramen Shop implements the proposed features: translation, rotation and scaling, a perspective camera, hierarchical modelling, ambient, diffuse and specular lighting, Gouraud and Phong shading, a skybox and procedural textures; the orthographic view is used for the HUD and flat shading was not implemented. Beyond the proposal it adds three kinds of shadow, glass refraction, ray tracing with a BVH, animated characters (a cyclist with inverse kinematics, a walking person and a cat with a state machine), a physically simulated wind chime, a procedural environment with my own grass and cobblestone textures, and a playable cooking game. For every feature this report gives the theory, the equations, the implementation and a screenshot, and all of it runs interactively from a single Visual Studio project with no external assets.

References

[1] B. T. Phong, “Illumination for computer generated pictures,” *Communications of the ACM* 18(6), 1975.

[2] J. F. Blinn, “Models of light reflection for computer synthesized pictures,” *SIGGRAPH*, 1977.

[3] H. Gouraud, “Continuous shading of curved surfaces,” *IEEE Transactions on Computers* C-20(6), 1971.

[4] T. Whitted, “An improved illumination model for shaded display,” *Communications of the ACM* 23(6), 1980.

[5] L. Williams, “Casting curved shadows on curved surfaces,” *SIGGRAPH*, 1978.

[6] W. Reeves, D. Salesin and R. Cook, “Rendering antialiased shadows with depth maps,” *SIGGRAPH*, 1987.

[7] C. Schlick, “An inexpensive BRDF model for physically-based rendering,” *Eurographics*, 1994.

[8] I. Sutherland and G. Hodgman, “Reentrant polygon clipping,” *Communications of the ACM* 17(1), 1974.

[9] D. Shreiner et al., *OpenGL Programming Guide* (the “Red Book”), Addison-Wesley; Khronos Group, *OpenGL and GLSL 1.20 specifications*.

[10] D. Ebert et al., *Texturing and Modeling: A Procedural Approach*, Morgan Kaufmann (value noise, fractal Brownian motion).